



Design and Implementation of IT/OT Integration in IIoT Applications

Patthanan Bhandhumanee

Faculty of Engineering, King Mongkut's University of Technology North Bangkok

Student ID : 6301011610279

Advisor: Dr.Ing Rawat Siripokapirom

Project II 010113942

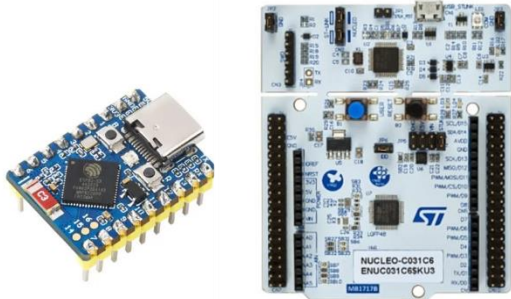
Examination Date 23 JUN 2026



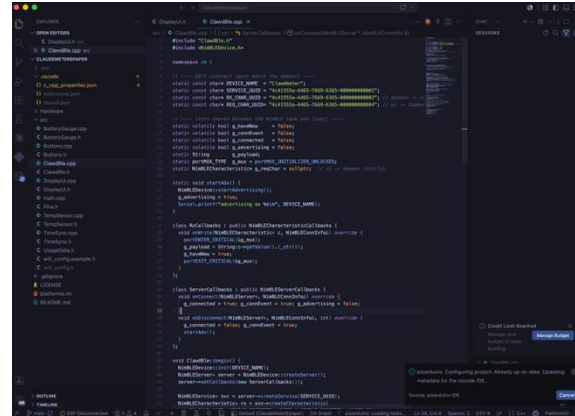
Inspiration

Embedded System & IOT

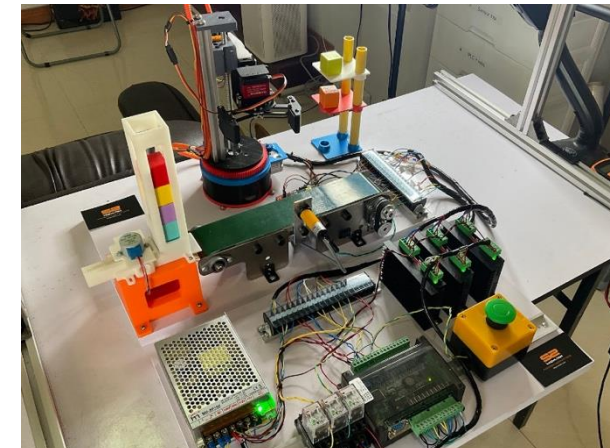
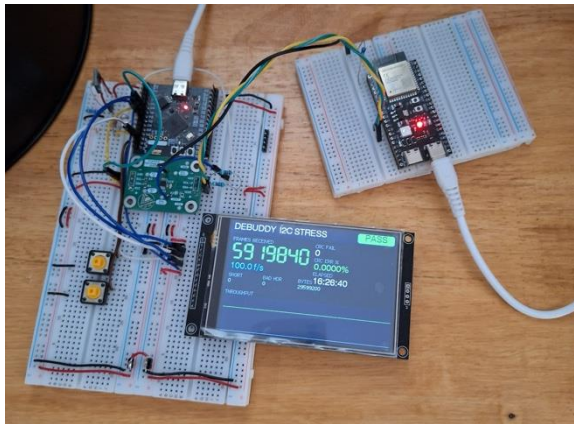
Automation



Microcontrollers



Programming

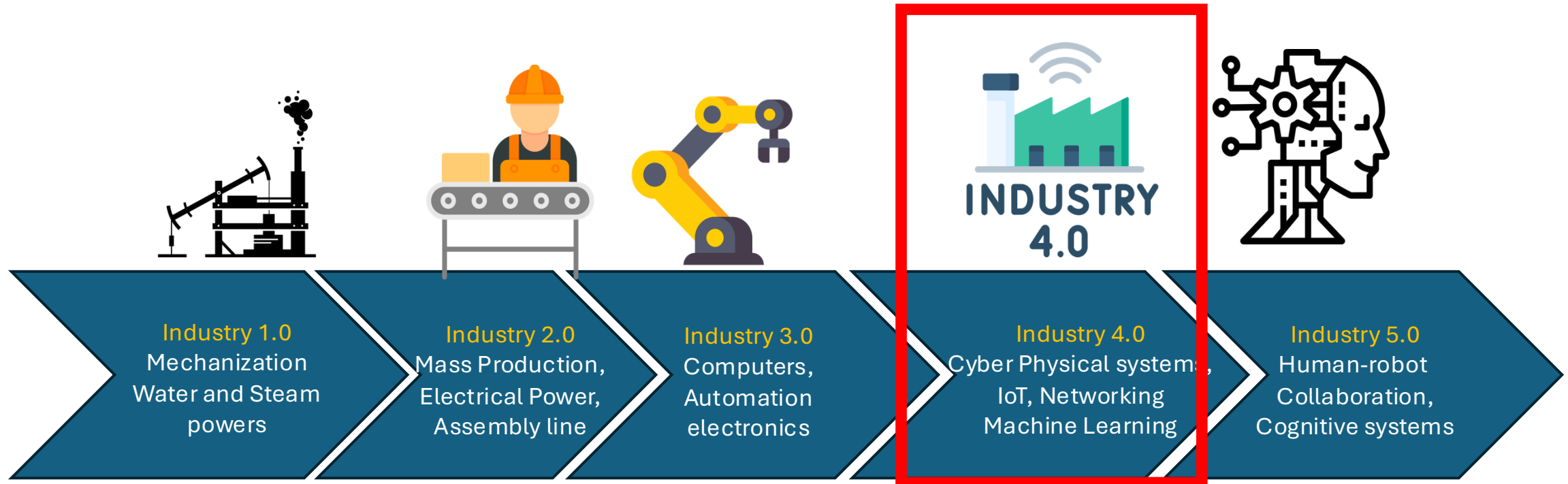




What if it all came in
one Lab Experiment Kit?



Industrial Revolutions

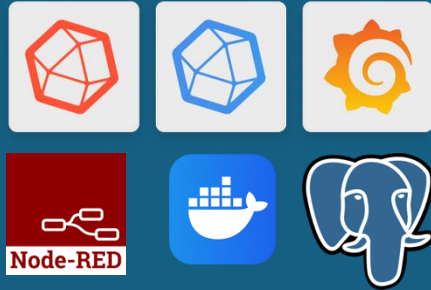


Information Technology
Operational Technology

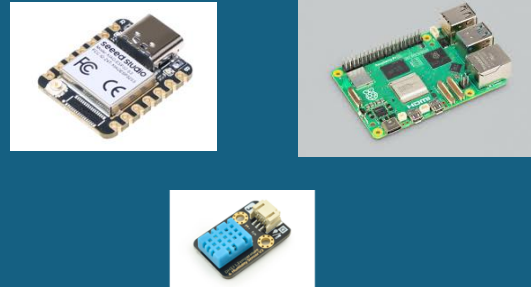


SOFTWARE, HARDWARE and PROTOCOLS

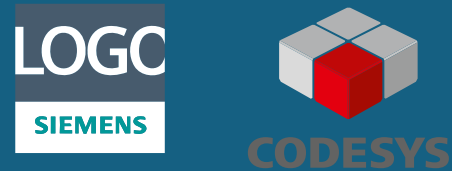
SOFTWARE



HARDWARE



PROTOCOLS





Objectives

Hardware Selection (Focusing on Low-Cost Hardware)

- SBC (Raspberry Pi4)
- MCU (ESP32S3)
- PLC (Siemens LOGO8.4)



SIEMENS

PROTOCOLS

- MODBUS RTU/TCP
- MQTT (websocket/over TCP)



SOFTWARE (Focusing on opensource software)

- Mosquitto
- TIG (Telegraf + InfluxDB + Grafana)
- NodeRED



Strengths & Weaknesses: PLC vs SBC vs MCU



PLC

PROS

- Hard/soft real-time, deterministic scan
- Rugged — built for harsh environments
- Native industrial I/O & protocols

CONS

- Needs proprietary software to program
- Slower cycle (depends on scan time)
- Higher cost per unit



SBC

PROS

- Full Linux OS — rich software ecosystem
- High clock speed & memory
- Easy networking, databases & AI

CONS

- Non-deterministic — no real-time guarantee
- Sensitive to harsh environments
- Higher power consumption



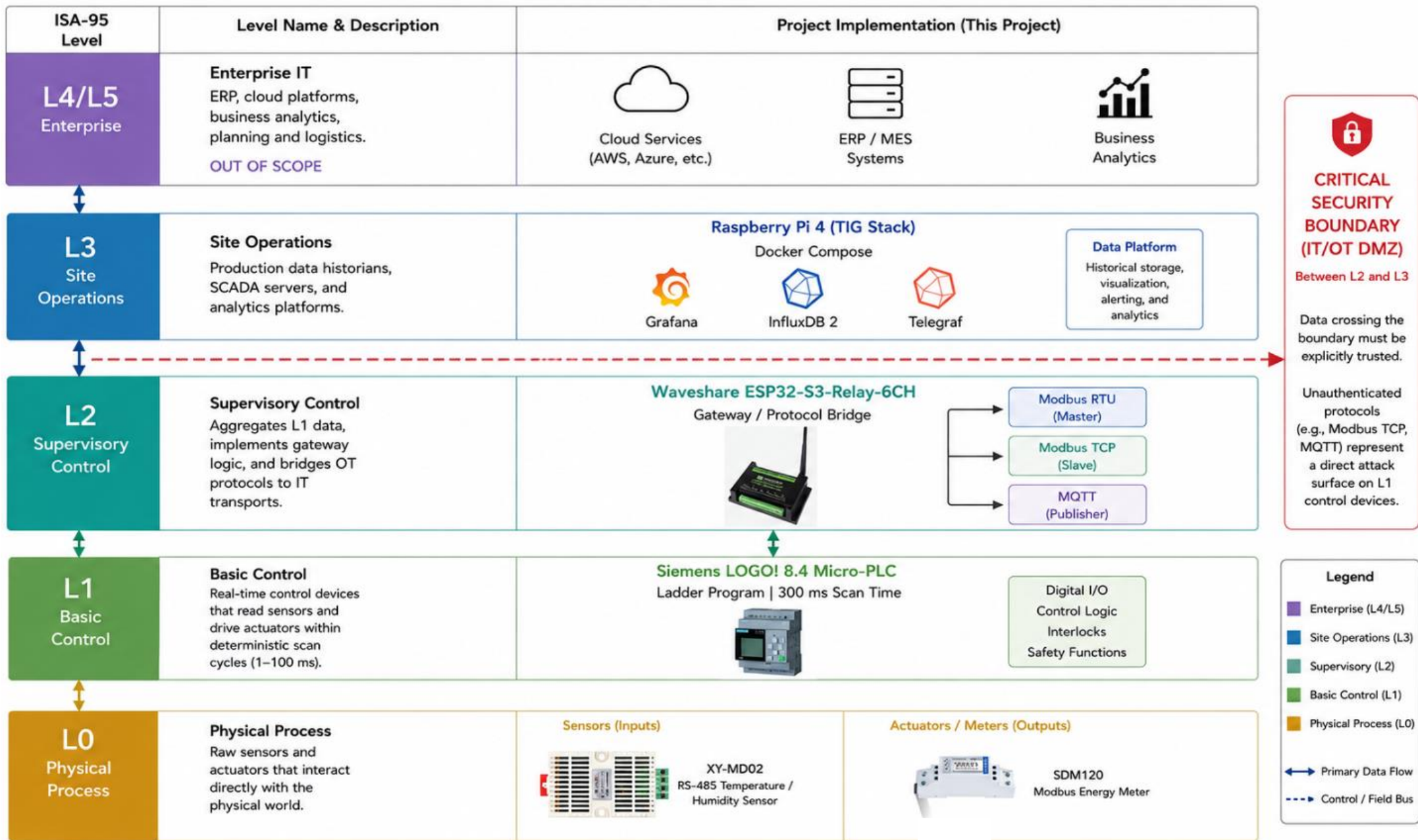
MCU

PROS

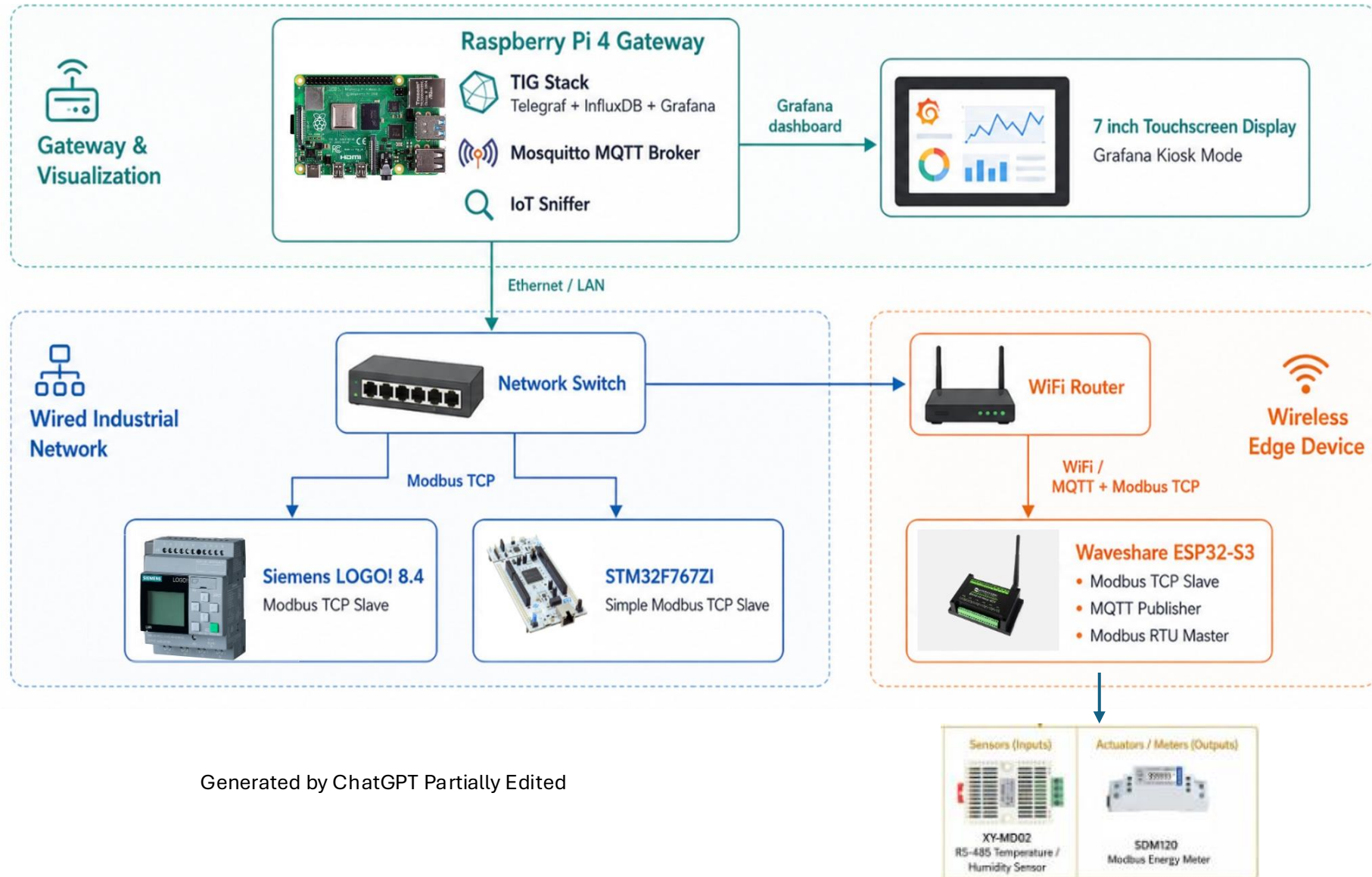
- Real-time with an RTOS
- Low cost & low power
- Flexible — many languages & frameworks

CONS

- Limited compute & memory
- Sensitive to harsh environments
- Slower than an SBC



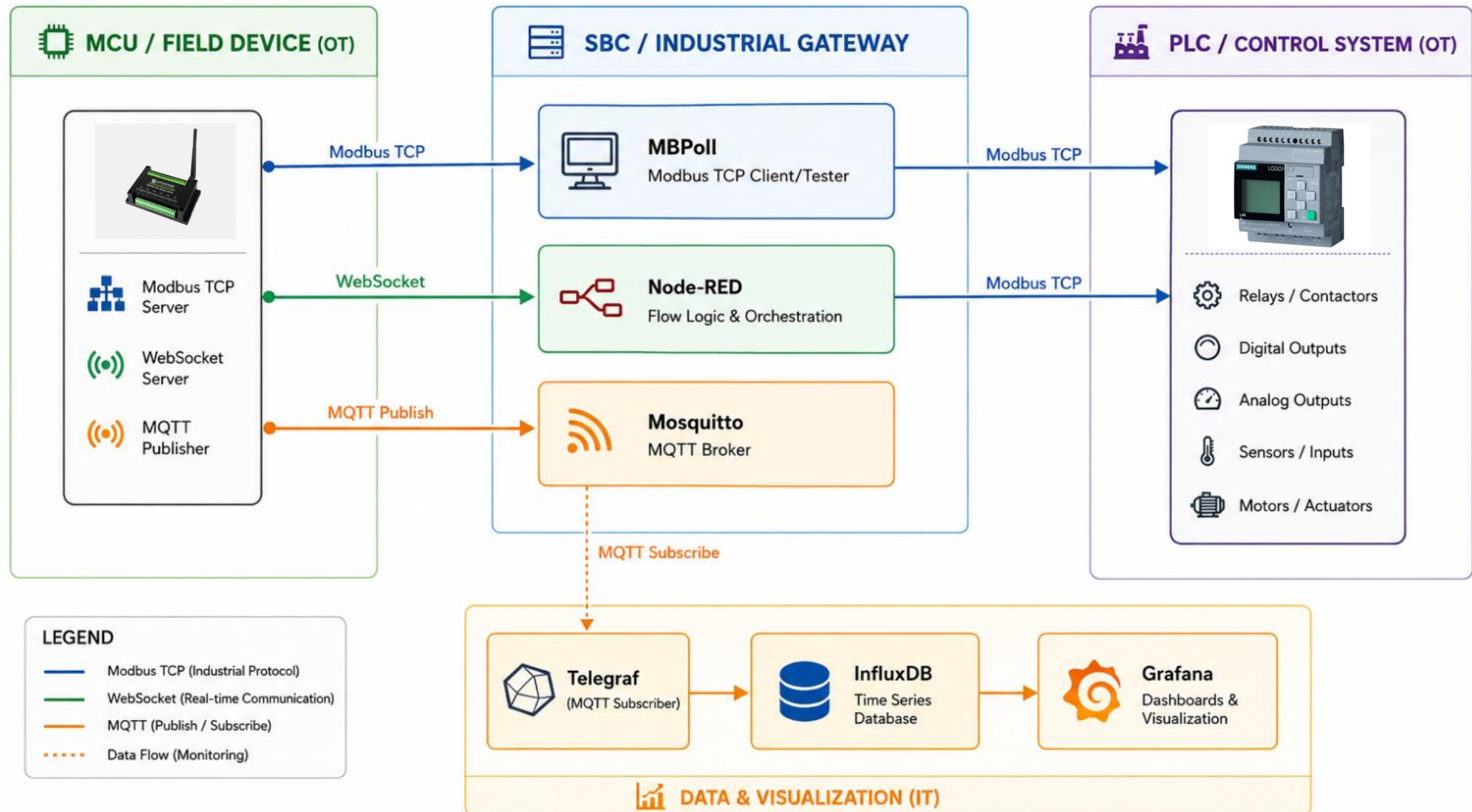
Proposed Hardware Architecture



Generated by ChatGPT Partially Edited

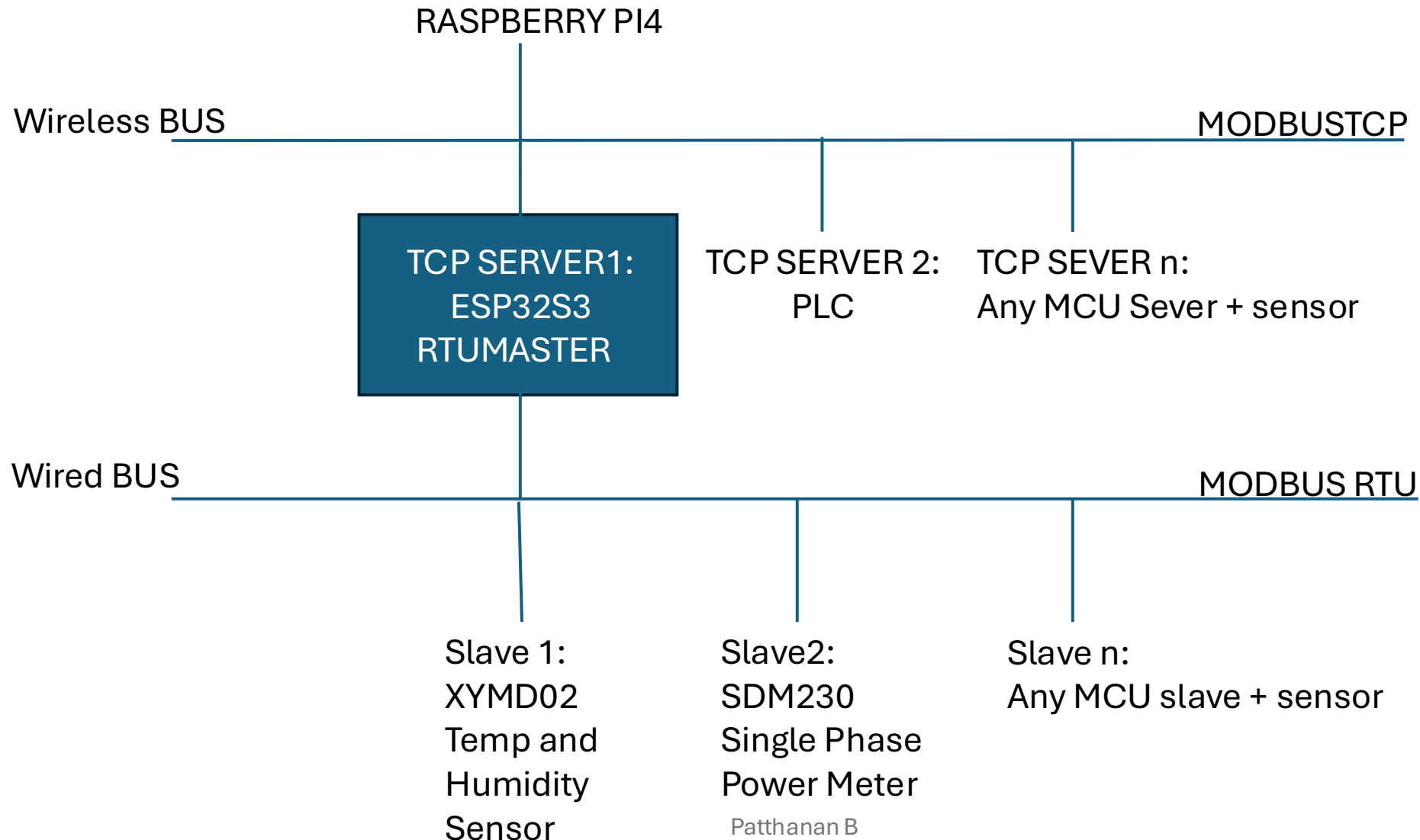


Software Architecture





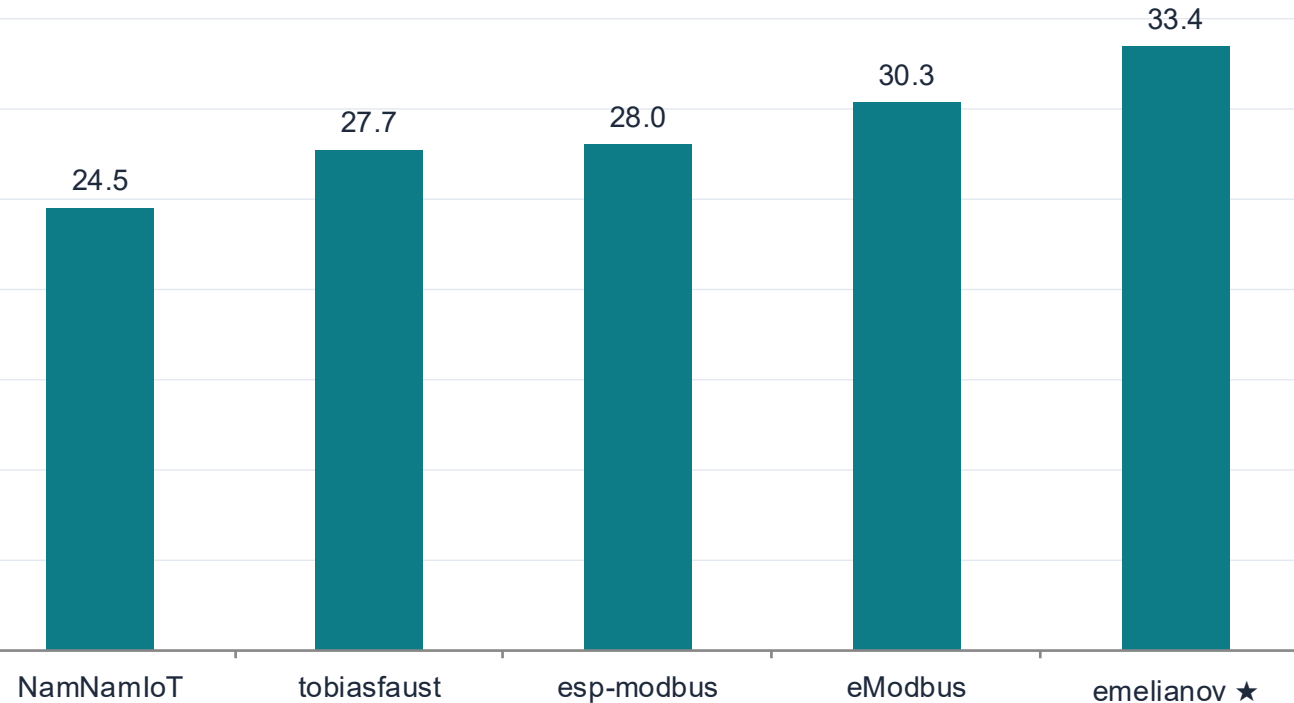
ESP32S3 Based MODBUS Gateway





MODBUS Gateway Libraries

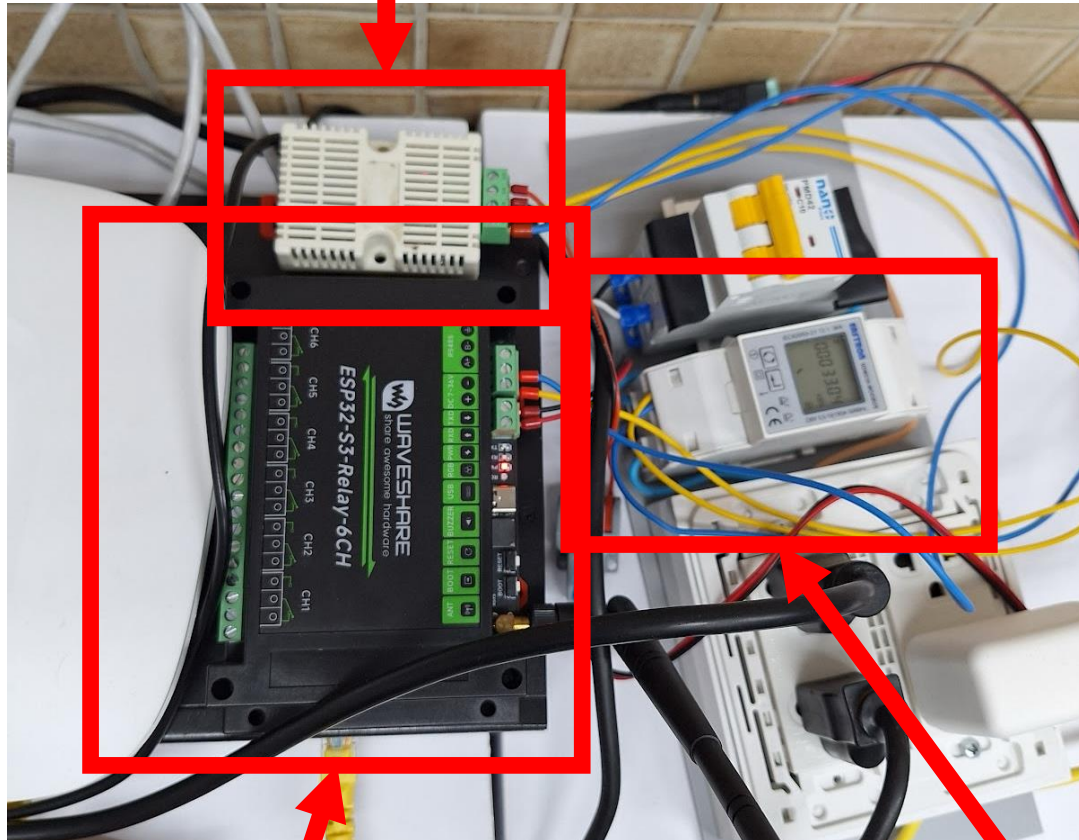
FC03 mean round-trip latency on real hardware (lower is better)



Library	Framework	FC3 mean (ms)	FC3 p95 (ms)	Max (ms)	Flash (KB)	RAM (KB)	All PASS
NamNamIoT / ModbusMaster	Arduino	24.5	30.6	30.8	301	22.3	20/20 ✓
tobiasfaust (raw RTU)	Arduino	27.7	34.2	34.2	300	22.0	20/20 ✓
vwetter (raw RTU)	Arduino	27.9	34.2	34.2	301	22.0	20/20 ✓
harihanv (raw RTU)	Arduino	28.0	34.2	34.2	300	22.0	24/24 ✓
maxx-ukoo / esp-modbus	ESP-IDF	27.8	34.4	34.5	270	14.4	24/24 ✓
esp-modbus (Espressif)	ESP-IDF	28.0	34.1	34.3	270	14.4	24/24 ✓
eModbus / eModbus	Arduino	30.3	36.0	36.0	384	22.9	20/20 ✓
zivillian / eModbus	Arduino	32.9	36.0	132.3 ⚠	384	22.9	20/20 ✓
emelianov / modbus-esp8266	Arduino	33.4	40.0	40.0	310	22.2	20/20 ✓ ★

Hardware Setup

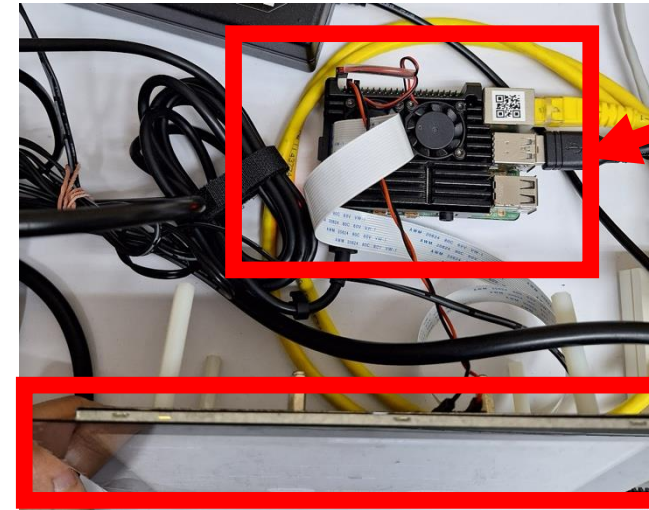
XYMD02



ESP32S3

SDM120

Raspberry Pi 4



Raspberry Pi Touch Screen

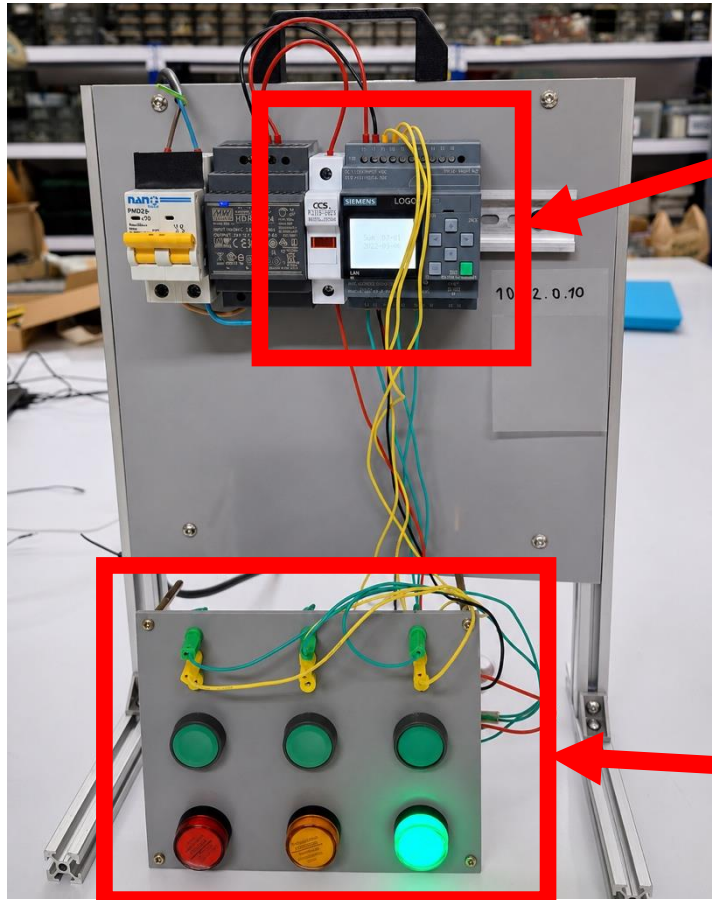




PLC experiment

- Basic I/O testing (Push button and Pilot Lamp)
- Manipulating I/O of PLC (MODBUSTCP SERVER) using
 - NodeRED
 - mbpoll (CLI)

Hardware



PLC: Siemens LOGO! 8.4

Modbus Poll Command:

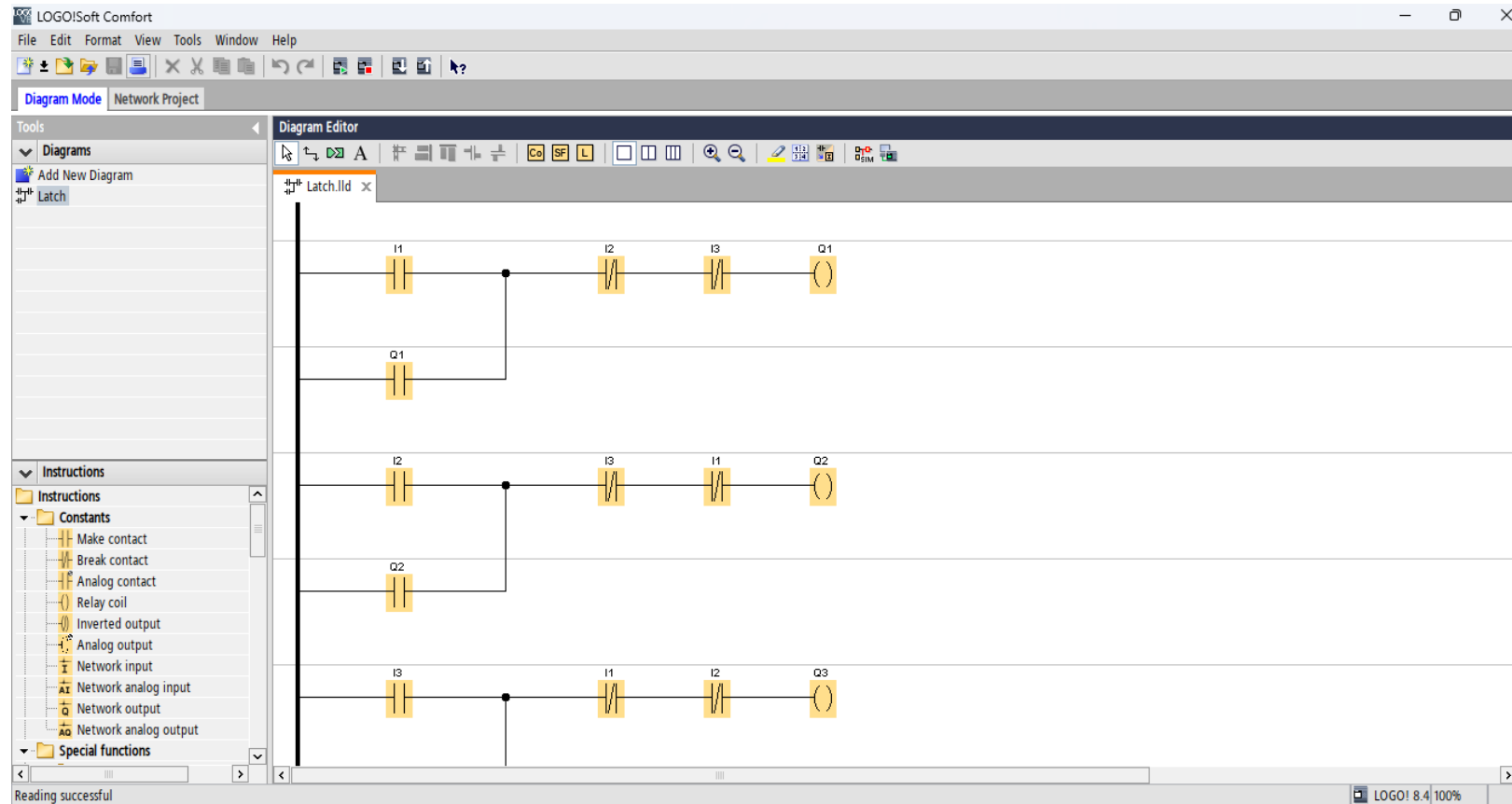
Write Q1 ON / OFF (FC05):

```
bash
mbpoll -m tcp -a 1 -t 0 -r 8193 192.168.0.3 1 # ON
mbpoll -m tcp -a 1 -t 0 -r 8193 192.168.0.3 0 # OFF
```

IO/Panel



Ladder Diagram





MODBUS TCP REGISTERS

LOGO! settings

Offline settings Online settings

General
Hardware type
I/O settings
I/O names
Program password
Power on
Message text
Additional info
Statistics
Comment
Modbus address space
Email Event Settings

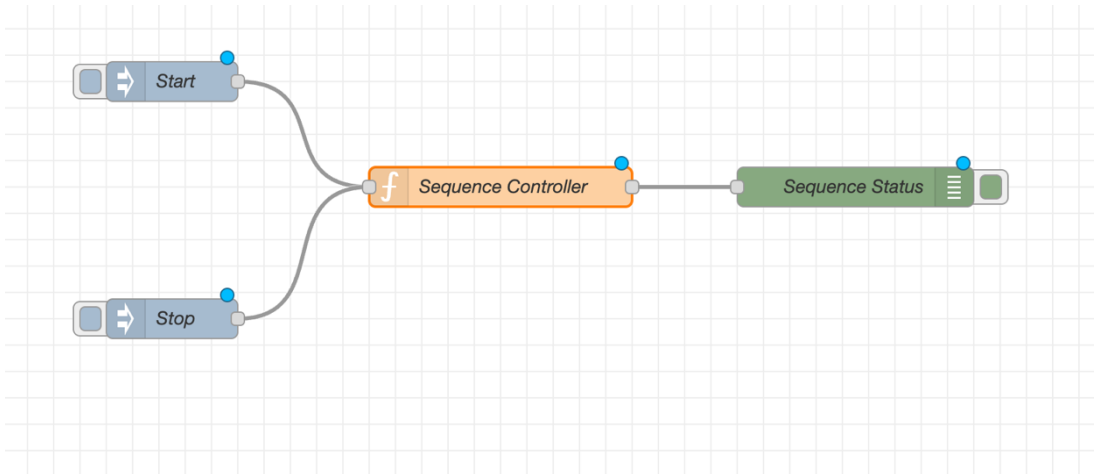
Modbus address space

Address Type	Range	Mapped Modbus Address	Direction	Unit
I	1 - 24	Discrete Input (DI) 1 - 24	R	bit
Q	1 - 20	Coil 8193 - 8212	R/W	bit
M	1 - 64	Coil 8257 - 8320	R/W	bit
V	0.0 - 850.7	Coil 1 - 6808	R/W	bit
AI	1 - 8	Input Register (IR) 1 - 8	R	word
VW	0 - 848	Holding Register (HR) 1 - 425	R/W	word
AQ	1 - 8	Holding Register (HR) 513 - 520	R/W	word
AM	1 - 64	Holding Register (HR) 529 - 592	R/W	word
VX	0.0 - 2047.7	Coil 16385 - 32768	R/W	bit
VR	0.0 - 511.7	Coil 32769 - 36864	R/W	bit
VXW	0 - 2046	Holding Register (HR) 801 - 1824	R/W	word
VRW	0 - 510	Holding Register (HR) 1825 - 2080	R/W	word

OK Cancel Help



NODERED: LED SEQUENCER

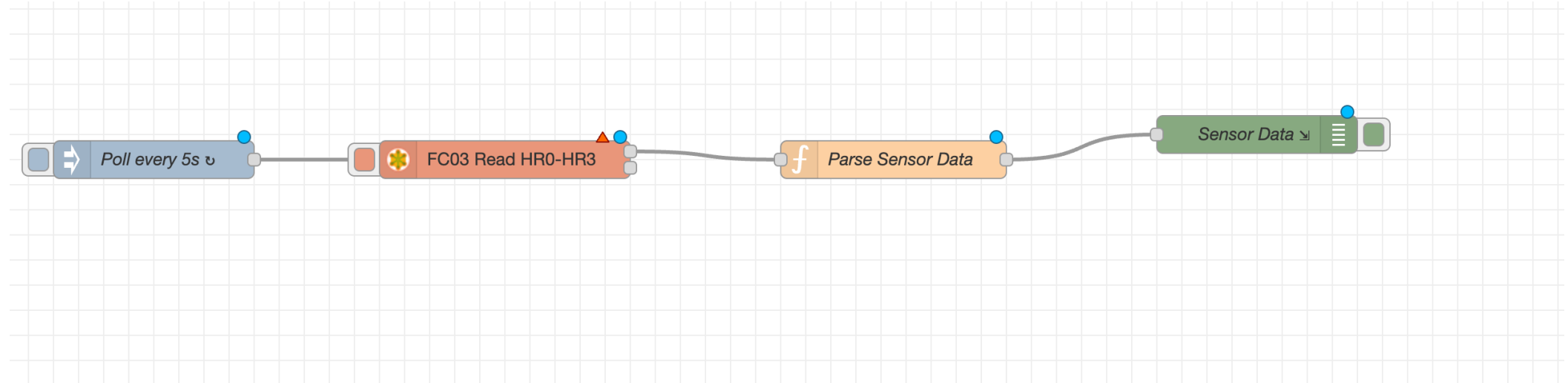


NodeRED flow for sending MODBUSTCP manipulating IO of Siemens LOGO8.4 to flash led in sequence

```
1 const interval = context.get('interval');
2
3 function closeConn() {
4   const c = context.get('conn');
5   if (c) { try { c.destroy(); } catch(e) {} context.set('conn', null); }
6 }
7
8 function writeFC5(client, address, on, tid) {
9   const buf = Buffer.alloc(12);
10  buf.writeUInt16BE(tid, 0); buf.writeUInt16BE(0, 2); buf.writeUInt16BE(6, 4);
11  buf[6] = 1; buf[7] = 0x05;
12  buf.writeUInt16BE(address, 8);
13  buf.writeUInt16BE(on ? 0xFF00 : 0x0000, 10);
14  client.write(buf);
15 }
16
17 // Q1=VX1.0 (coil 8), Q2=coil 8193, Q3=coil 8194 (0-based)
18 const Q = [8, 8193, 8194];
19
20 if (msg.payload === 'start') {
21   if (interval) clearInterval(interval);
22   closeConn();
23   let step = 0;
24   let tid = 0;
25   const client = net.createConnection({ host: '10.42.0.10', port: 502 }, () => {
26     node.status({ fill: 'green', shape: 'dot', text: 'Q1-Q2-Q3' });
27     const newInterval = setInterval(() => {
28       for (let i = 0; i < 3; i++) {
29         tid = (tid + 1) & 0xFFFF;
30         writeFC5(client, Q[i], i === step ? 1 : 0, tid);
31       }
32       node.send({ payload: 'Q' + (step + 1) + ' on' });
33       step = (step + 1) % 3;
34     }, 500);
35     context.set('interval', newInterval);
36   });
37   client.on('error', (err) => {
38     node.error('Connection error: ' + err.message);
39     node.status({ fill: 'red', shape: 'ring', text: err.message });
40     closeConn();
41   });
42   client.on('close', () => {
43     const iv = context.get('interval');
44     if (iv) { clearInterval(iv); context.set('interval', null); }
45     context.set('conn', null);
46   });
47   context.set('conn', client);
48
49 } else if (msg.payload === 'stop') {
```



NODERED: ESP32 MODBUS TCP SERVER



Polling the ESP32 Registers every 5 seconds



How do we measure the
performance of those IoT devices?

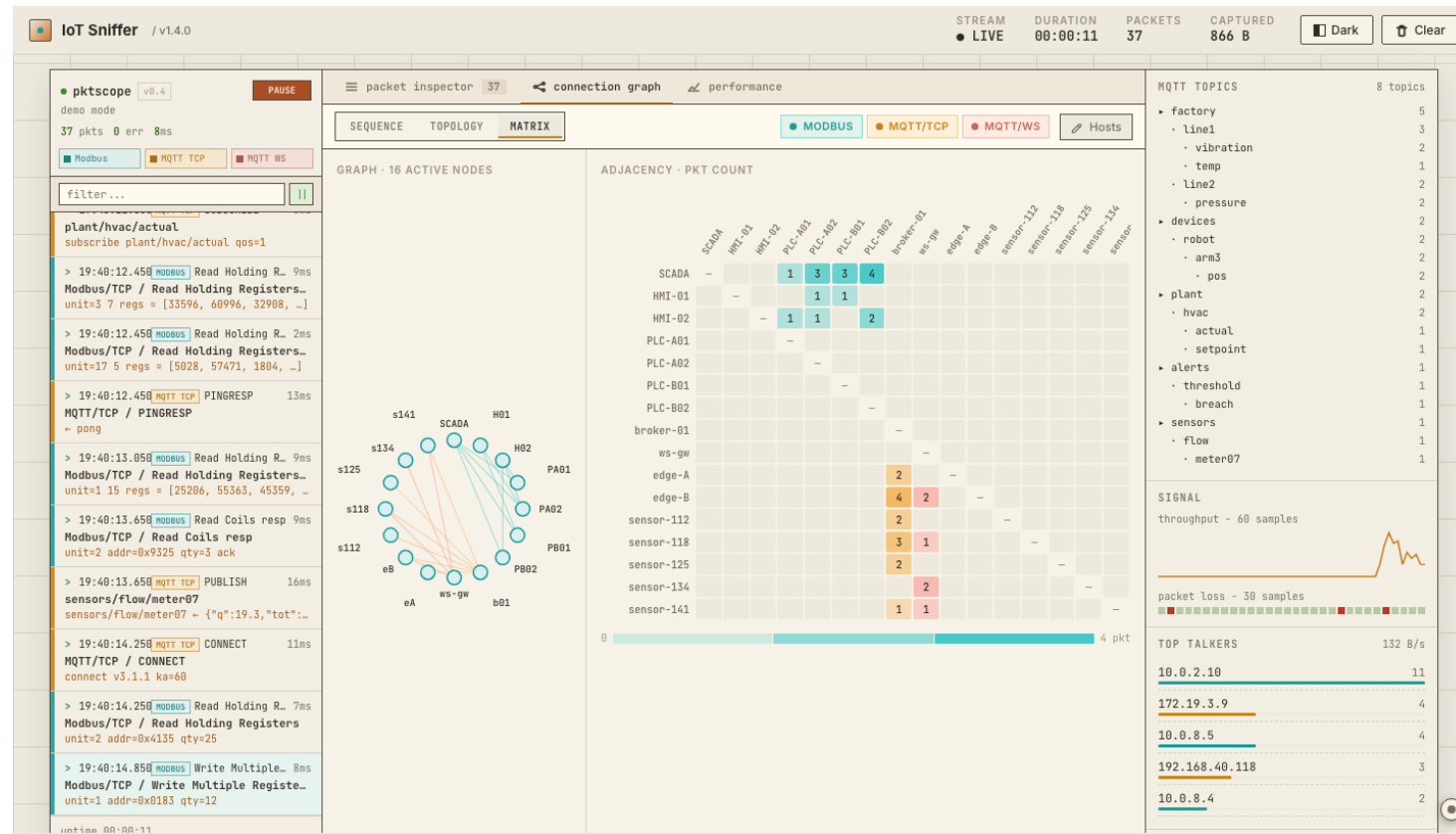


IoT sniffer Features

- Can sniff the IoT packets of
 - MODBUS TCP default port 502
 - MQTT (websocket) default port 1883
 - MQTT (over TCP) default port 8083
- Visualize connection between devices on system
- Visualize topology of the system



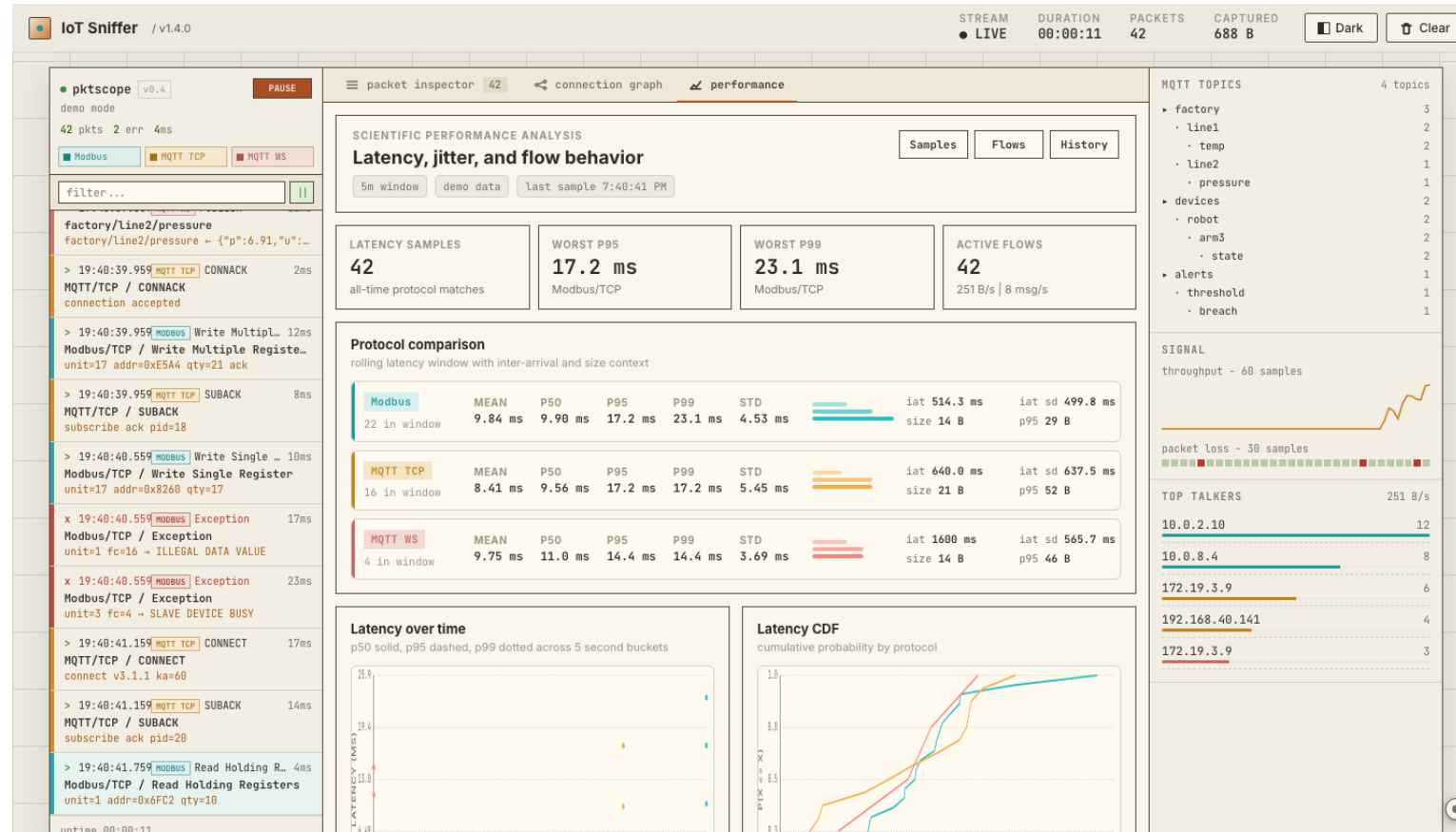
IoT Sniffer



Connection Diagram And Topology



IoT Sniffer



Measuring Throughput of devices



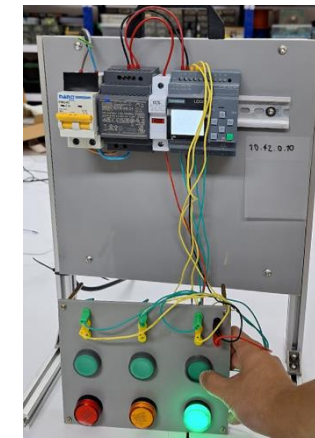
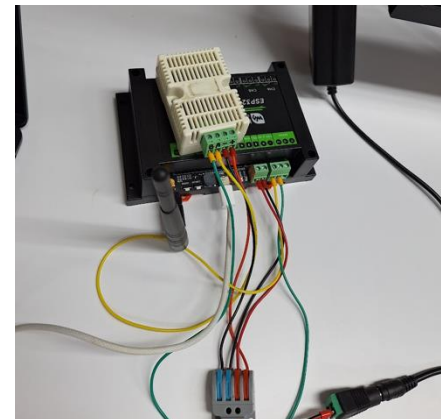
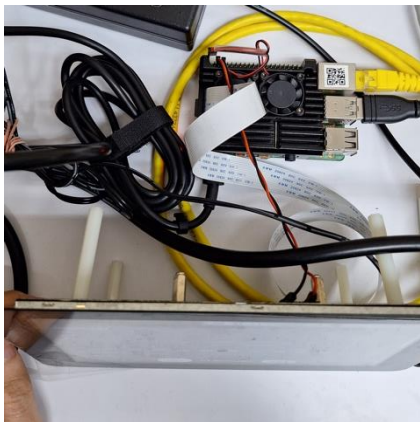
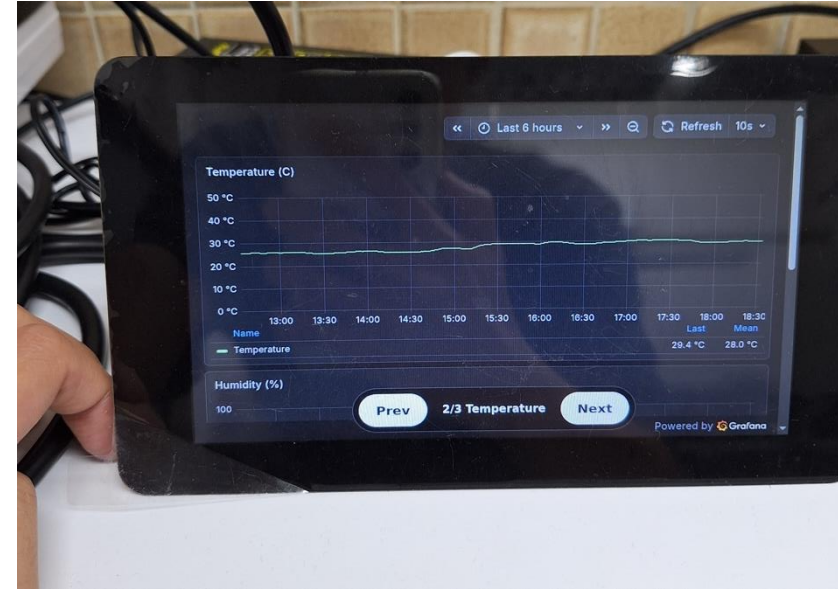
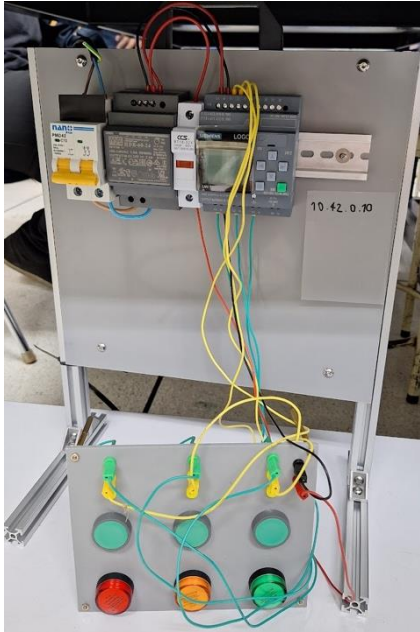
Conclusions

1. created IIoT Laboratory platform that consist of 3 main catagories of hardwares SBC, MCU and PLC.
2. Demonstrated integration between IT and OT devices with emphasis on open-source software stack.
3. Evaluated and ESP32S3 based MODBUS Gateway libraries.
4. Develop IoT-Sniffer to observe network activity and measure network performance of IoT devices



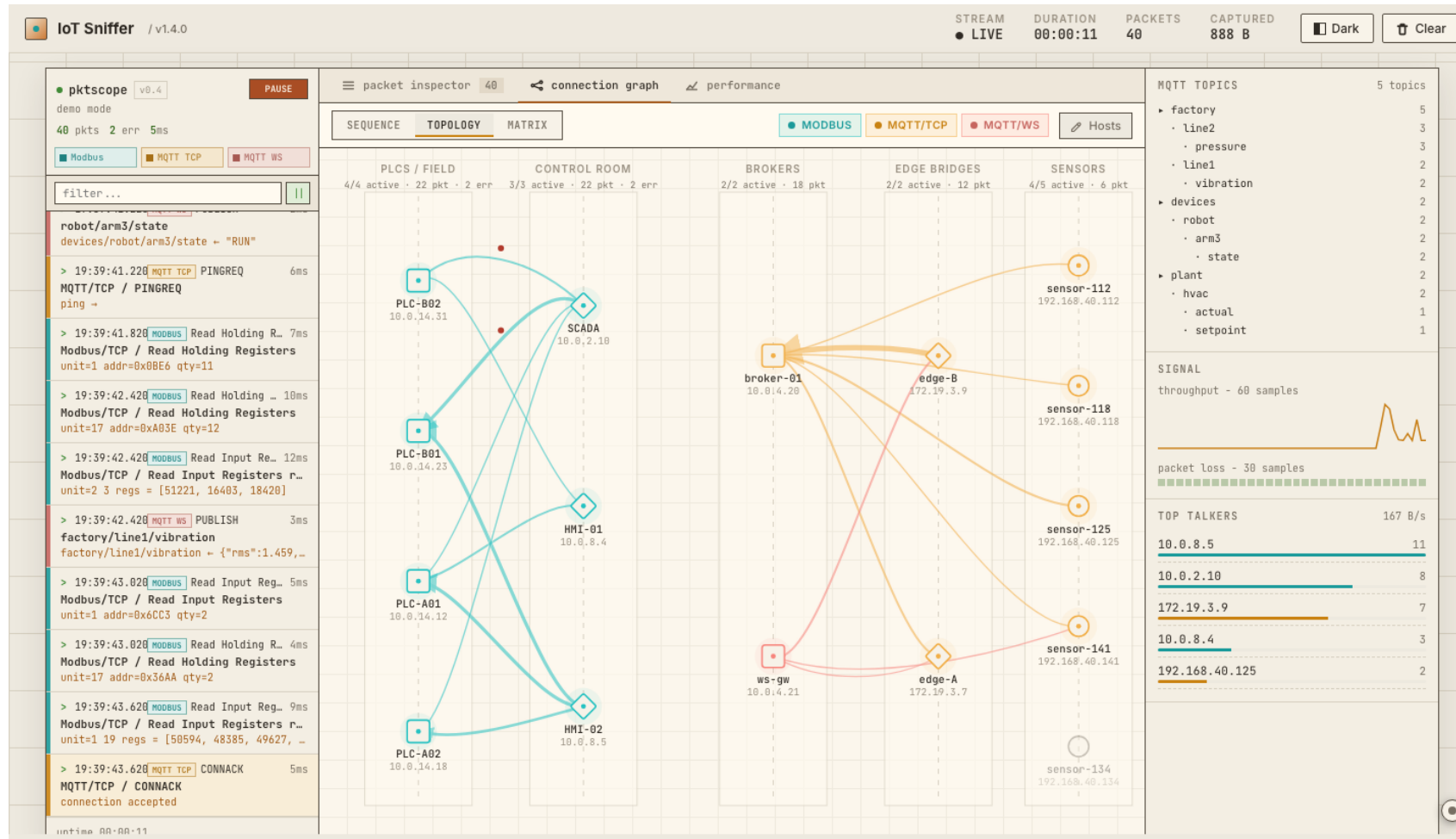
THANK YOU
FOR YOUR ATTENTION
Q&A

Real Hardware Pictures





IoT sniffer





IoT sniffer

